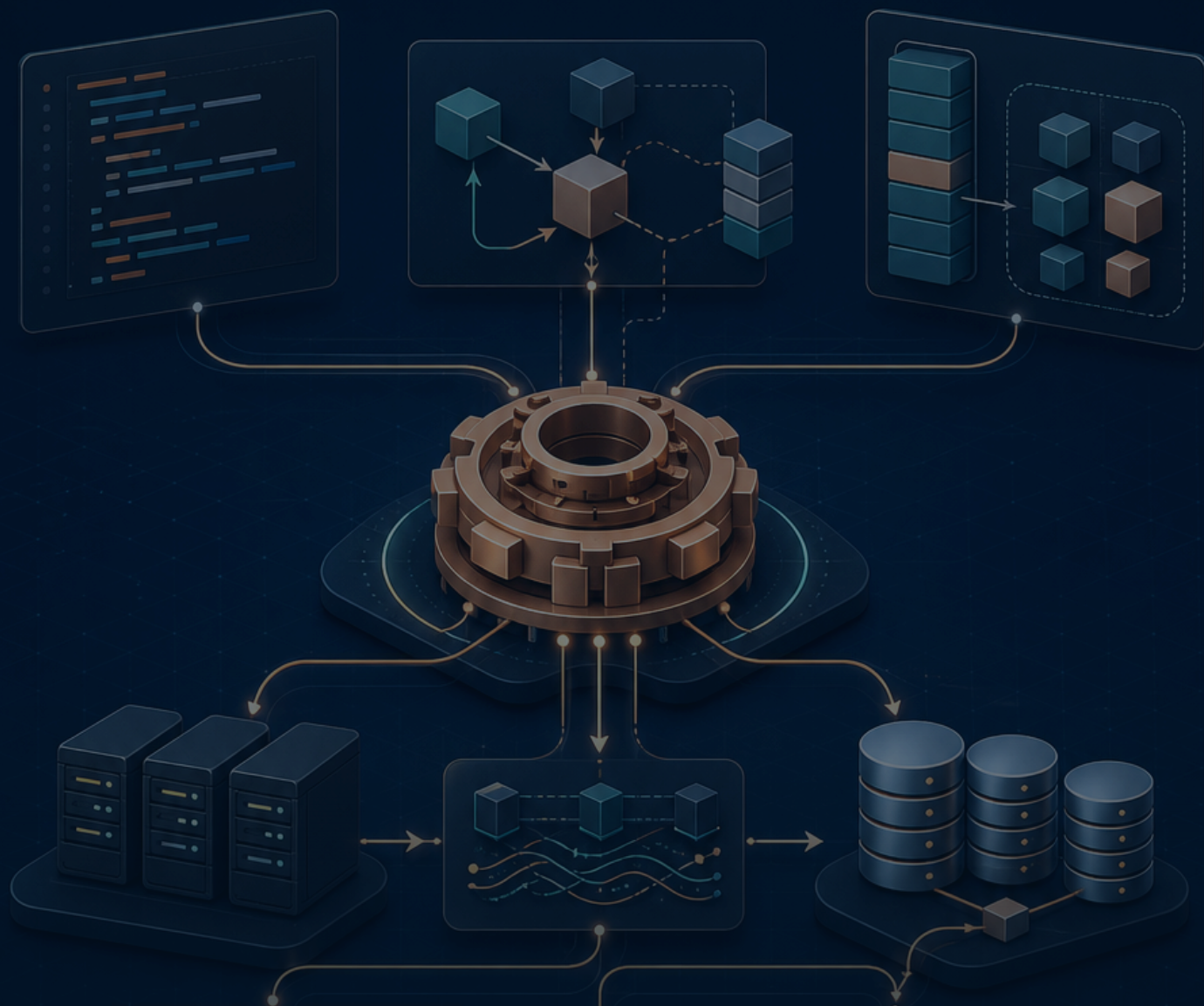




VOLUME 10 / RUST BACKEND MASTERY

RustNote — CRUD・検索・UI完全実装

一覧、作成、詳細、編集、競合、toggle、削除、検索、pagination、通知、アクセシビリティを縦切りで完成させる

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

各コード片の直後にブラウザ表示、HTTP結果、DB変化、失敗ケースを並べ、何が起きたか追えるようにします。

到達目標

- ✓ 全CRUDをowner条件付きで実装できる
- ✓ validationとPRGを正しく使える
- ✓ 競合・pagination・検索を安全に設計できる
- ✓ 画面・HTTP・DB・a11yを通して検証できる

学習地図

01 一覧表示・空状態・所有者query

GET /notesはcurrent userを解決し、user_id付きqueryでNoteを新着順に取得し、空状態またはcard一覧をSSRします。layoutとpageでcurrent_userを共有memoizeします。

02 作成form・validation・PRG

POST /notesでFormをparseし、title/bodyを検証してからINSERTします。error時は400で入力と項目errorを再表示し、成功時は303でGET /notesへ戻ります。

03 詳細・path ID・not found

GET /notes/{note_id}は0.5.0の#[path_param]でi64へparseし、正数検証後、idとuser_idで一件取得します。存在なしと他人所有は同じ404にして列挙を防ぎます。

04 編集・事前表示・競合

GET editはowner noteをformへ入れ、POST editは再検証してUPDATEします。updated_atまたはversionで楽観lockを行うと、別tabの更新を上書きせず409を返せます。

05 toggle・原子的更新・冪等性

done切替をread→反転→writeの三段階にすると競合で更新を失います。SQL一文のCASEで原子的に切り替え、owner条件とRETURNINGを付けます。

06 削除・確認・hard/soft delete

削除はGETではなくPOST/DELETEで行い、owner条件を含めます。誤操作回復や監査が必要ならdeleted_atのsoft delete、法的消去なら関連データを含むhard deleteを選びます。

07 検索・filter・並び替え

query parameterをtyped structへparseし、許可したsortとstatusだけをenumへ変換します。検索語はbindし、LIKE wildcardを利用者が意図しない場合はescapeします。

08 pagination・cursor・件数

OFFSETは簡単ですが深いpageで遅く更新中に重複・欠落しやすいです。created_atとidのcursor paginationは決定的順序を使い、次cursorをopaqueにします。

09 flash・error summary・PRG後の通知

redirect後の成功通知は一回限りのflashとしてsessionまたは短命cookieへ保存し、次GETで取り出して削除します。errorはredirectせず400で同じformへ表示します。

10 responsive UI・a11y・総合CRUD試験

CRUD画面をmobile、keyboard、screen reader、高contrastで確認します。見た目のscreenshotだけでなくHTTP、DB、認可、focus、accessible nameまでe2eで検証します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

auth guard

STEP 2

owner SELECT

STEP 3

empty state

STEP 4

list component

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

一覧表示・空状態・所有者query

UNIT 01・CONCEPT

一覧表示・空状態・所有者query

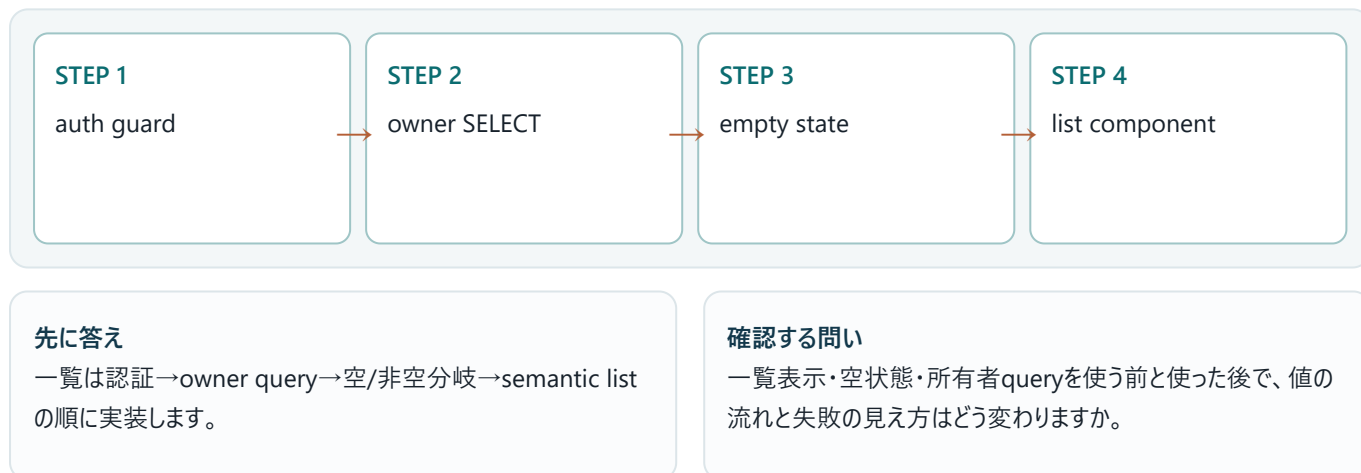
GET /notesはcurrent userを解決し、user_id付きqueryでNoteを新着順に取得し、空状態またはcard一覧をSSRします。layoutとpageでcurrent_userを共有memoizeします。

この単元の到達点

- auth guard
- owner SELECT
- empty state
- list component

なぜバックエンドで必要か

SELECTにuser_idとLIMIT、決定的ORDER BYを含めます。view/password/session型を渡しません。



UNIT 01・MENTAL MODEL

一覧表示・空状態・所有者queryを図でつかむ



GET /notesはcurrent userを解決し、user_id付きqueryでNoteを新着順に取得し、空状態またはcard一覧をSSRします。layoutとpageでcurrent_userを共有memoizeします。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	auth guard	auth guard — 入力と前提を確認し、境界を曖昧にしない。
2	owner SELECT	owner SELECT — 値がどの順で変換されるかを追跡する。
3	empty state	empty state — 失敗を再現し、型またはログから原因を特定する。
4	list component	list component — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

一覧表示・空状態・所有者queryとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

一覧表示・空状態・所有者queryだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[page("/notes")]async fn index(cx:&Cx)->Result{let user=require_user(cx).await?;let notes=repo(cx).list(user.id,50).await?;view!{<main><h1>"ノート"</h1>if notes.is_empty(){<p>"最初のノートを作りましょう"</p>}else{<ul>for note in notes{note_card(note)}</ul>}}</main>}}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[page("/notes")]async fn index(cx:&Cx)->Result{let user=require_user(cx).</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
一覧表示・空状態・所有者query	入力、処理、出力を分離して読みます。
一覧表示・空状態・所有者query	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がauth guardの条件を満たすか確認する。
- ✓ owner SELECTを実行し、途中の型と状態を追う。
- ✓ empty stateで失敗を成功経路から分離する。
- ✓ list componentにより、出力と副作用を検証する。

型を声に出す 一覧表示・空状態・所有者queryに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

一覧表示・空状態・所有者query

見出し、作成form、filter、空状態または自分のNoteカードが新着順で表示されます。

観察ポイント

- auth guard
- owner SELECT
- empty state

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[query_params(error=bad_request)]struct Q{status:Option<String>}
let status=query_params::<Q>(cx)?.status.as_deref();let notes=repo.list(user.id,status,50).await?;
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01 · BACKEND APPLICATION

バックエンドのどこで使うか

SELECTにUser_idとLIMIT、決定的ORDER BYを含めます。viewへpassword/session型を渡しません。



リクエスト側

- auth guardを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- owner SELECTを小さな責務へ分割
- empty stateをResultとログで表現
- list componentをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	auth guardに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	owner SELECTの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	empty stateまたはlist componentの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

一覧表示・空状態・所有者queryを再現して切り分ける

一覧表示・空状態・所有者queryの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: auth guard
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: owner SELECT
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: empty state
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: list component

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	auth guard	auth guard — 入力と前提を確認し、境界を曖昧にしない。
2	owner SELECT	owner SELECT — 値がどの順で変換されるかを追跡する。
3	empty state	empty state — 失敗を再現し、型またはログから原因を特定する。
4	list component	list component — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 一覧は認証→owner query→空/非空分岐→semantic listの順に実装します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 ・ PRACTICE

手を動かす演習

未完了だけを絞るquery parameterを追加し、空状態文言もfilterに合わせてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[query_params(error=bad_request)]struct Q{status:Option<String>}
let status=query_params::<Q>(cx)?.status.as_deref();let notes=repo.list(user.id,status,50).await?;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 一覧は認証→owner query→空/非空分岐→semantic listの順に実装します。
- ✓ 一覧表示・空状態・所有者queryとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

作成form・validation・PRG

UNIT 02 ・ CONCEPT

作成form・validation・PRG

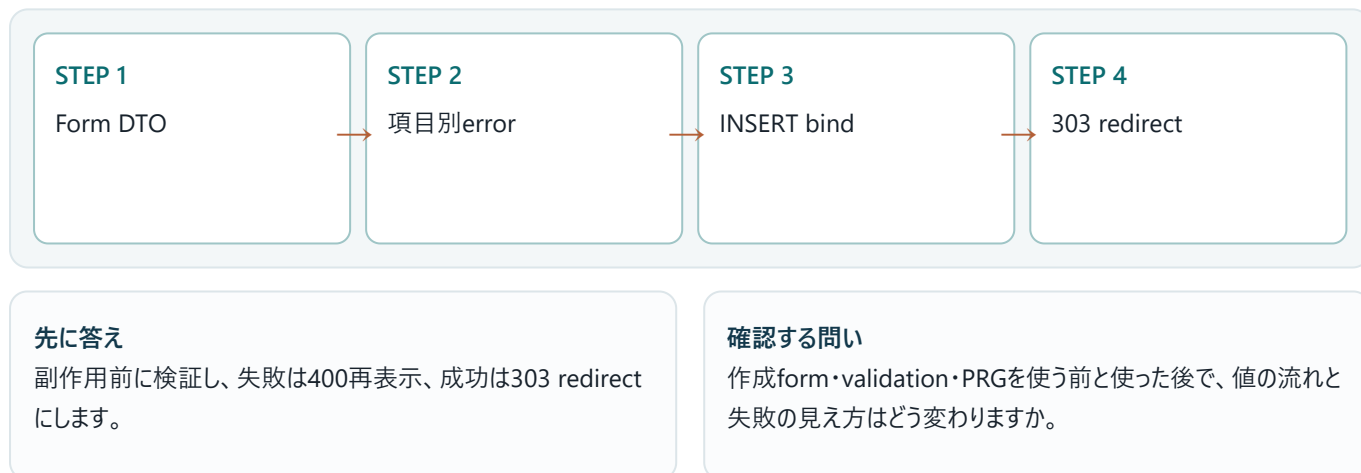
POST /notesでFormをparseし、title/bodyを検証してからINSERTします。error時は400で入力と項目errorを再表示し、成功時は303でGET /notesへ戻ります。

この単元の到達点

- Form DTO
- 項目別error
- INSERT bind
- 303 redirect

なぜバックエンドで必要か

validation前にDBへ触れず、DB CHECKとUNIQUE競合もAppErrorへ変換します。二重送信はPRGと一意キーで抑えます。



UNIT 02 ・ MENTAL MODEL

作成form・validation・PRGを図でつかむ



POST /notesでFormをparseし、title/bodyを検証してからINSERTします。error時は400で入力と項目errorを再表示し、成功時は303でGET /notesへ戻ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Form DTO	Form DTO — 入力と前提を確認し、境界を曖昧にしない。
2	項目別error	項目別error — 値がどの順で変換されるかを追跡する。
3	INSERT bind	INSERT bind — 失敗を再現し、型またはログから原因を特定する。
4	303 redirect	303 redirect — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

作成form・validation・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02・MINIMUM CODE

まず動く最小コード

作成form・validation・PRGだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[route(POST "/notes")]async fn create(cx:&Cx,Form(input):Form<CreateNote>)->Result<SeeOther>{let
user=require_user(cx).await?;let
cmd=CreateNoteCommand::try_from(input)?;repo(cx).insert(user.id,&cmd).await?;Ok(see_other("/notes"))}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[route(POST "/notes")]async fn create(cx:&Cx,Form(input):Form<CreateNote></code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
作成form・validation・PRG	入力、処理、出力を分離して読みます。
作成form・validation・PRG	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がForm DTOの条件を満たすか確認する。
- ✓ 項目別errorを実行し、途中の型と状態を追う。
- ✓ INSERT bindで失敗を成功経路から分離する。
- ✓ 303 redirectにより、出力と副作用を検証する。

型を声に出す 作成form・validation・PRGに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

作成form・validation・PRG

空titleでは入力欄の下にエラーが出てbodyは保持されます。成功後はURLが/notesになり、新しいカードが一度だけ出ます。

観察ポイント

- Form DTO
- 項目別error
- INSERT bind

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[test]fn title_boundaries(){assert!(Title::parse(&"あ".repeat(80)).is_ok());assert!(Title::parse(&"あ".repeat(81)).is_err());assert!(Title::parse(" ").is_err());}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

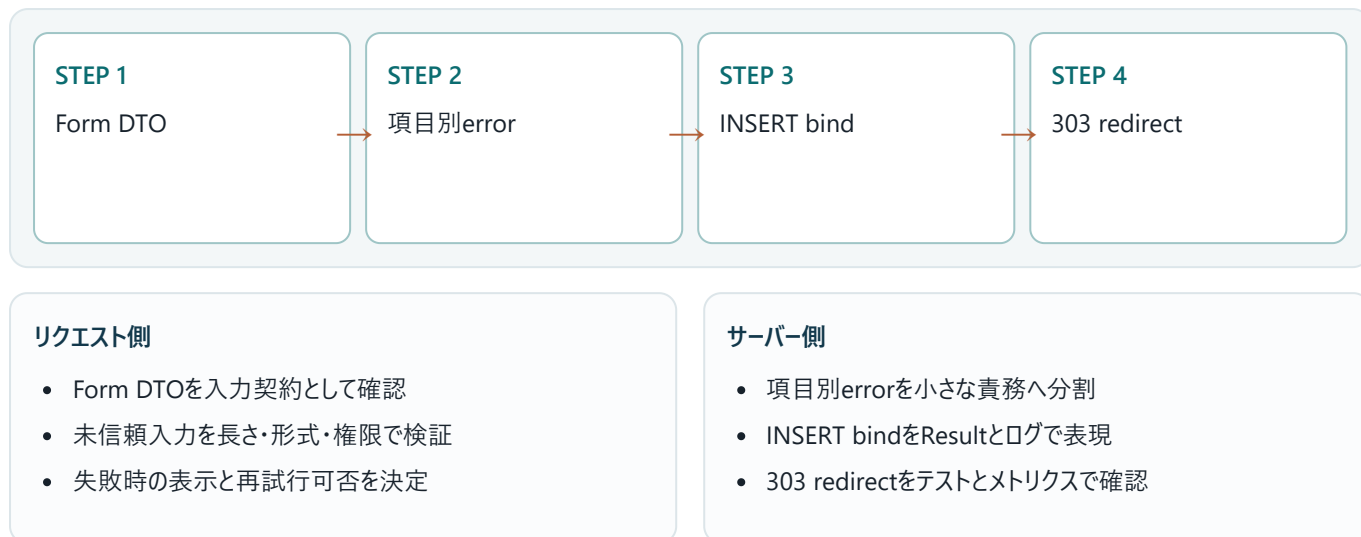
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

validation前にDBへ触れず、DB CHECKとUNIQUE競合もAppErrorへ変換します。二重送信はPRGと一意キーで抑えます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Form DTOに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	項目別errorの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	INSERT bindまたは303 redirectの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

作成form・validation・PRGを再現して切り分ける

作成form・validation・PRGの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Form DTO
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 項目別error
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: INSERT bind
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 303 redirect

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Form DTO	Form DTO — 入力と前提を確認し、境界を曖昧にしない。
2	項目別error	項目別error — 値がどの順で変換されるかを追跡する。
3	INSERT bind	INSERT bind — 失敗を再現し、型またはログから原因を特定する。
4	303 redirect	303 redirect — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 副作用前に検証し、失敗は400再表示、成功は303 redirectにします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

title 80文字境界、日本語、body 5000文字境界をtestしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[test]fn title_boundaries(){assert!(Title::parse(&"あ".repeat(80)).is_ok());assert!(Title::parse(&"あ".repeat(81)).is_err());assert!(Title::parse(" ").is_err());}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 副作用前に検証し、失敗は400再表示、成功は303 redirectにします。
- ✓ 作成form・validation・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

詳細・path ID・not found

UNIT 03 · CONCEPT

詳細・path ID・not found

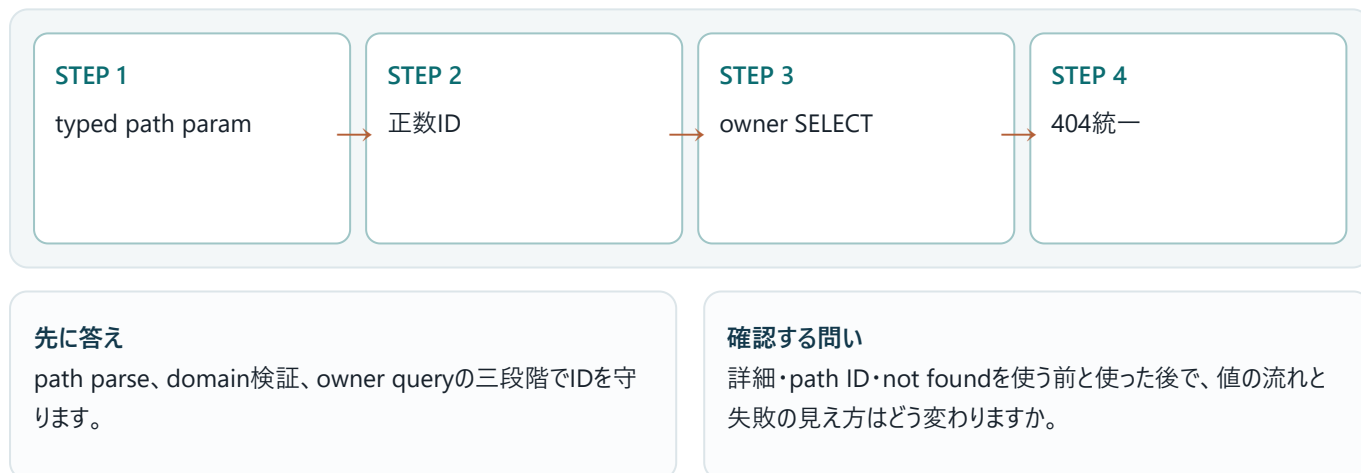
GET /notes/{note_id}は0.5.0の#[path_param]でi64へparseし、正数検証後、idとuser_idで一件取得します。存在なしと他人所有は同じ404にして列挙を防ぎます。

この単元の到達点

- typed path param
- 正数ID
- owner SELECT
- 404統一

なぜバックエンドで必要か

ID parse成功は認可成功を意味しません。DB queryまでowner条件を運び、error bodyにDBや他user情報を出しません。



UNIT 03 · MENTAL MODEL

詳細・path ID・not foundを図でつかむ



GET /notes/{note_id}は0.5.0の#[path_param]でi64へparseし、正数検証後、idとuser_idで一件取得します。存在なしと他人所有は同じ404にして列挙を防ぎます。図では左から右へ処理を追います。各箱の入口では入力のと信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	typed path param	typed path param — 入力と前提を確認し、境界を曖昧にしない。
2	正数ID	正数ID — 値がどの順で変換されるかを追跡する。
3	owner SELECT	owner SELECT — 失敗を再現し、型またはログから原因を特定する。
4	404統一	404統一 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

詳細・path ID・not foundとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE まず動く最小コード

詳細・path ID・not foundだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[path_param(error=bad_request)]struct NoteId(i64);
#[page("/notes/{note_id}")]async fn show(cx:&Cx)->Result{let user=require_user(cx).await?;let
id=*path_param::<NoteId>(cx)?;let note=repo(cx).find_owned(user.id,id.0).await?.ok_or_not_found()?;view!
{<article><h1>(&note.title)</h1><p>(&note.body)</p></article>}}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[path_param(error=bad_request)]struct NoteId(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[page("/{notes}/{note_id}")]async fn show(cx:&Cx)->Result{let user=require_</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
詳細・path ID・not found	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がtyped path paramの条件を満たすか確認する。
- ✓ 正数IDを実行し、途中の型と状態を追う。
- ✓ owner SELECTで失敗を成功経路から分離する。
- ✓ 404統一により、出力と副作用を検証する。

型を声に出す 詳細・path ID・not foundに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

詳細・path ID・not found

自分のNoteはtitle/body/edit linkを表示し、不正IDは一般的なエラーページになります。

観察ポイント

- typed path param
- 正数ID
- owner SELECT

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
assert_eq!(get("/notes/abc").status(),400);assert_eq!(get("/notes/0").status(),400);assert_eq!(get_as(user_a,format!("{}/notes/{}",note_b.id)).status(),404);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

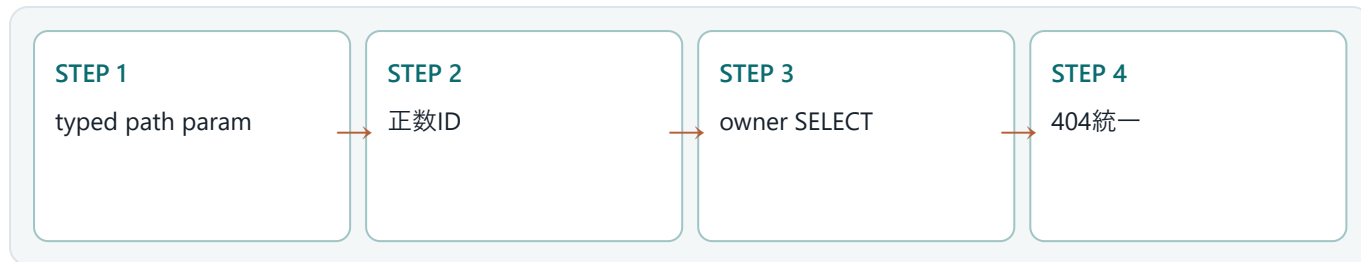
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03・BACKEND APPLICATION

バックエンドのどこで使うか

ID parse成功は認可成功を意味しません。DB queryまでowner条件を運び、error bodyにDBや他user情報を出しません。



リクエスト側

- typed path paramを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 正数IDを小さな責務へ分割
- owner SELECTをResultとログで表現
- 404統一をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	typed path paramに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	正数IDの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	owner SELECTまたは404統一の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

詳細・path ID・not foundを再現して切り分ける

詳細・path ID・not foundの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: typed path param
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 正数ID
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: owner SELECT
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 404統一

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	typed path param	typed path param — 入力と前提を確認し、境界を曖昧にしない。
2	正数ID	正数ID — 値がどの順で変換されるかを追跡する。
3	owner SELECT	owner SELECT — 失敗を再現し、型またはログから原因を特定する。
4	404統一	404統一 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 path parse、domain検証、owner queryの三段階でIDを守ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

0と負数を400、他人IDを404にするintegration testを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
assert_eq!(get("/notes/abc").status(), 400); assert_eq!(get("/notes/0").status(), 400); assert_eq!(get_as(user_a, format!("/notes/{}", note_b.id)).status(), 404);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ path parse、domain検証、owner queryの三段階でIDを守ります。
- ✓ 詳細・path ID・not foundとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

編集・事前表示・競合

UNIT 04 · CONCEPT

編集・事前表示・競合

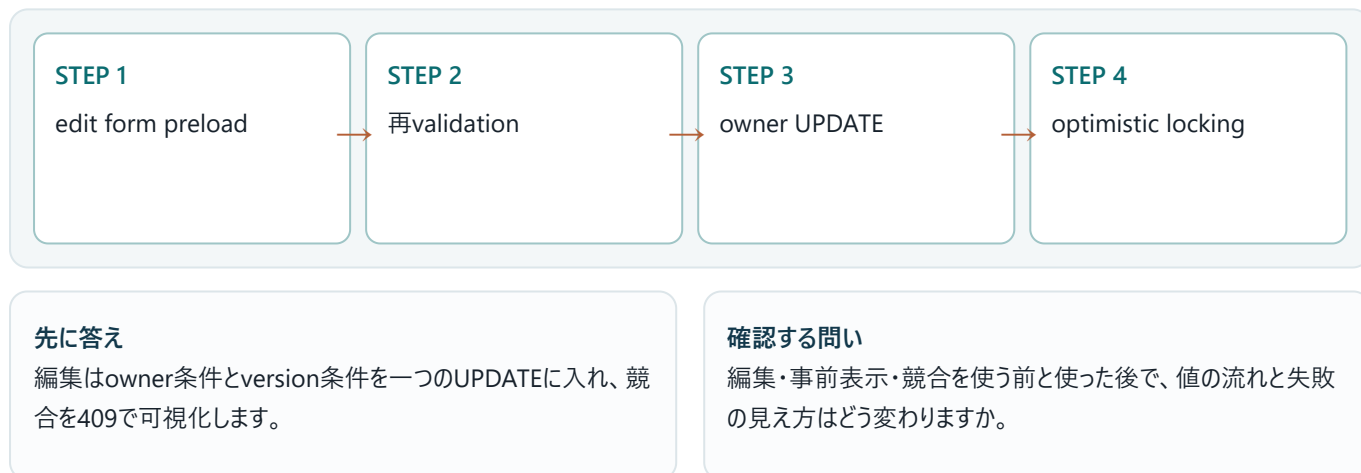
GET editはowner noteをformへ入れ、POST editは再検証してUPDATEします。updated_atまたはversionで楽観lockを行うと、別tabの更新を上書きせず409を返せます。

この単元の到達点

- edit form preload
- 再validation
- owner UPDATE
- optimistic locking

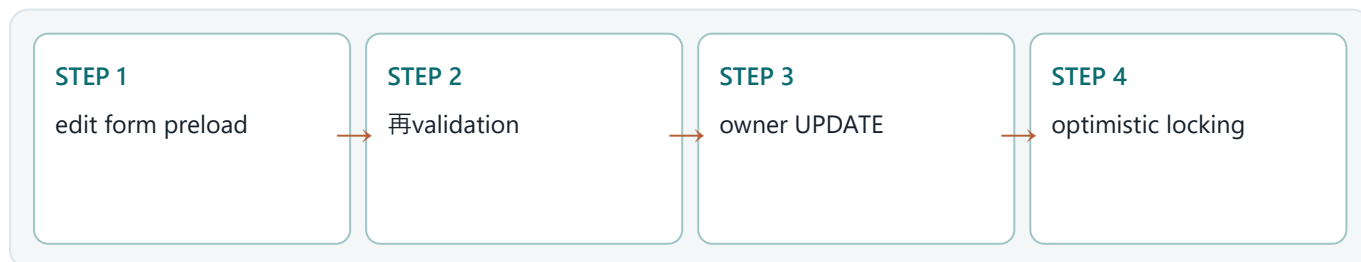
なぜバックエンドで必要か

GET時のversionをhidden fieldへ持たせますが、user_idはhidden値を信用せずsessionから取ります。



UNIT 04 · MENTAL MODEL

編集・事前表示・競合を図でつかむ



GET editはowner noteをformへ入れ、POST editは再検証してUPDATEします。updated_atまたはversionで楽観lockを行うと、別tabの更新を上書きせず409を返せます。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	edit form preload	edit form preload — 入力と前提を確認し、境界を曖昧にしない。
2	再validation	再validation — 値がどの順で変換されるかを追跡する。
3	owner UPDATE	owner UPDATE — 失敗を再現し、型またはログから原因を特定する。
4	optimistic locking	optimistic locking — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

編集・事前表示・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE まず動く最小コード

編集・事前表示・競合だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
UPDATE notes SET title=?1,body=?2,version=version+1,updated_at=CURRENT_TIMESTAMP WHERE id=?3 AND user_id=?4 AND version=?5 RETURNING version;
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
UPDATE notes SET title=?1,body=?2,version=version+1,updated_at=CURRENT_TIM	失敗時は早期returnし、呼び出し元へエラーを伝播します。

コード／場所	役割と理由
編集・事前表示・競合	入力、処理、出力を分離して読みます。
編集・事前表示・競合	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がedit form preloadの条件を満たすか確認する。
- ✓ 再validationを実行し、途中の型と状態を追う。
- ✓ owner UPDATEで失敗を成功経路から分離する。
- ✓ optimistic lockingにより、出力と副作用を検証する。

型を声に出す 編集・事前表示・競合に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

編集・事前表示・競合

競合時は『別の場所で更新されました』と最新内容へのlink、入力中内容を表示します。

観察ポイント

- edit form preload
- 再validation
- owner UPDATE

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
let a=load(note).await;let b=load(note).await;assert!(update(note,a.version,"A").await.is_ok());assert!(matches!(update(note,b.version,"B").await,Err(AppError::Conflict)));
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

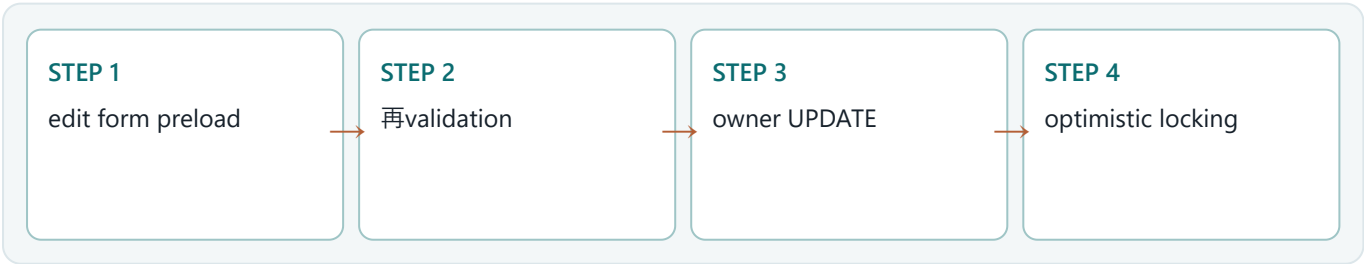
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

GET時のversionをhidden fieldへ持たせますが、user_idはhidden値を信用せずsessionから取ります。



リクエスト側

- edit form preloadを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 再validationを小さな責務へ分割
- owner UPDATEをResultとログで表現
- optimistic lockingをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	edit form preloadに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	再validationの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	owner UPDATEまたはoptimistic lockingの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

編集・事前表示・競合を再現して切り分ける

編集・事前表示・競合の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: edit form preload
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 再validation
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: owner UPDATE
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: optimistic locking

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	edit form preload	edit form preload — 入力と前提を確認し、境界を曖昧にしない。
2	再validation	再validation — 値がどの順で変換されるかを追跡する。
3	owner UPDATE	owner UPDATE — 失敗を再現し、型またはログから原因を特定する。
4	optimistic locking	optimistic locking — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 編集はowner条件とversion条件を一つのUPDATEに入れ、競合を409で可視化します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

二つのtabがversion=3を編集し、先の更新だけ成功、後を409にするtestを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
let a=load(note).await;let b=load(note).await;assert!(update(note,a.version,"A").await.is_ok());assert!(matches!(update(note,b.version,"B").await,Err(AppError::Conflict)));
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 編集はowner条件とversion条件を一つのUPDATEに入れ、競合を409で可視化します。
- ✓ 編集・事前表示・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

toggle・原子的更新・冪等性

UNIT 05 · CONCEPT

toggle・原子的更新・冪等性

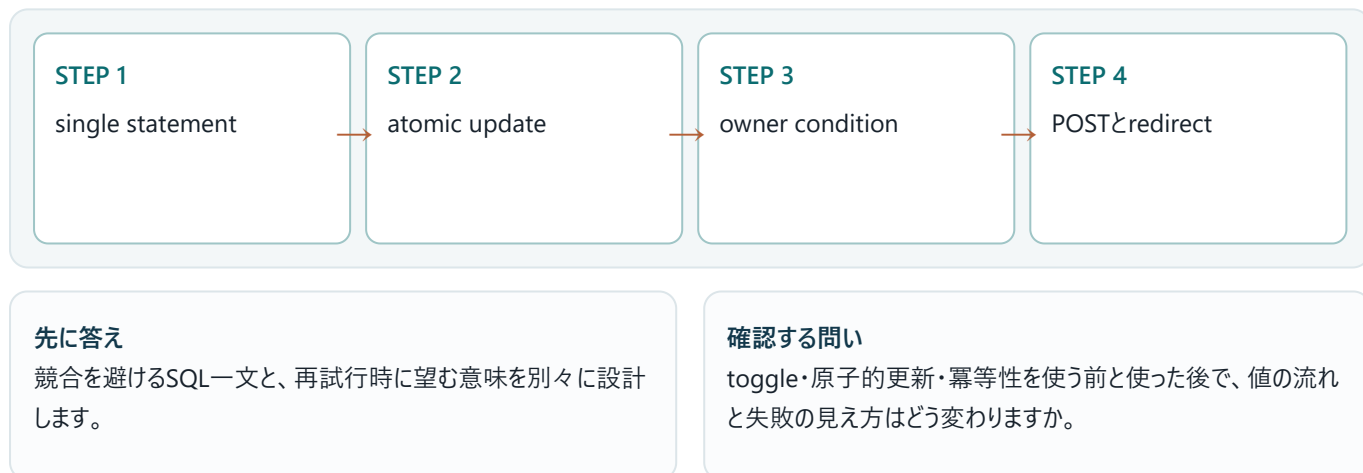
done切替をread→反転→writeの三段階にすると競合で更新を失います。SQL一文のCASEで原子的に切り替え、owner条件とRETURNINGを付けます。

この単元の到達点

- single statement
- atomic update
- owner condition
- POSTとredirect

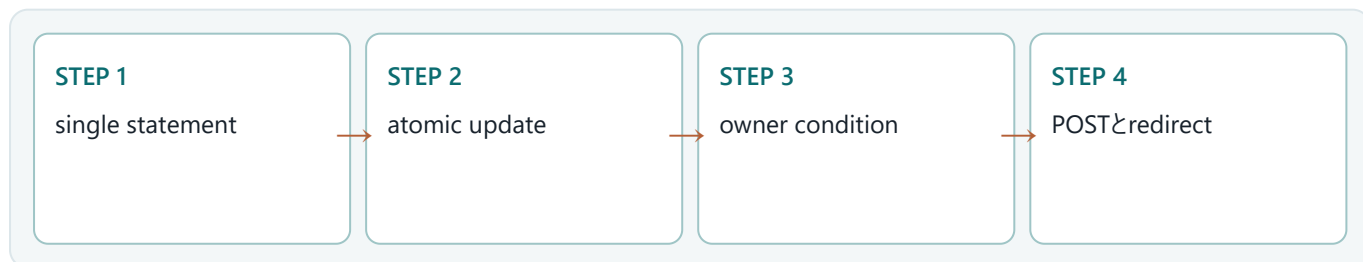
なぜバックエンドで必要か

toggleは冪等ではないためretryで二回実行すると元へ戻ります。明示的なset-done true/false routeやidempotency keyも検討します。



UNIT 05 · MENTAL MODEL

toggle・原子的更新・冪等性を図でつかむ



done切替をread→反転→writeの三段階にすると競合で更新を失います。SQL一文のCASEで原子的に切り替え、owner条件とRETURNINGを付けます。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	single statement	single statement — 入力と前提を確認し、境界を曖昧にしない。
2	atomic update	atomic update — 値がどの順で変換されるかを追跡する。
3	owner condition	owner condition — 失敗を再現し、型またはログから原因を特定する。
4	POSTとredirect	POSTとredirect — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

toggle・原子的更新・冪等性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE
まず動く最小コード

toggle・原子的更新・冪等性だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
UPDATE notes SET done=CASE done WHEN 0 THEN 1 ELSE 0 END,version=version+1,updated_at=CURRENT_TIMESTAMP
WHERE id=?1 AND user_id=?2 RETURNING done;
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
UPDATE notes SET done=CASE done WHEN 0 THEN 1 ELSE 0 END,version=version+1	失敗時は早期returnし、呼び出し元へエラーを伝播します。

コード／場所	役割と理由
toggle・原子的更新・冪等性	入力、処理、出力を分離して読みます。
toggle・原子的更新・冪等性	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がsingle statementの条件を満たすか確認する。
- ✓ atomic updateを実行し、途中の型と状態を追う。
- ✓ owner conditionで失敗を成功経路から分離する。
- ✓ POSTとredirectにより、出力と副作用を検証する。

型を声に出す toggle・原子的更新・冪等性に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

toggle・原子的更新・冪等性

完了操作後にcardのstyleと状態文字が変わり、screen readerにも『完了』が伝わります。

観察ポイント

- single statement
- atomic update
- owner condition

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
UPDATE notes SET done=?1 WHERE id=?2 AND user_id=?3 AND done<>?1 RETURNING done;
// 0 rowsでも既に期待状態ならsuccessとして扱う設計
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

toggleは冪等ではないためretryで二回実行すると元へ戻ります。明示的なset-done true/false routeや idempotency keyも検討します。



リクエスト側

- single statementを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- atomic updateを小さな責務へ分割
- owner conditionをResultとログで表現
- POSTとredirectをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	single statementに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	atomic updateの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	owner conditionまたはPOSTとredirectの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

toggle・原子的更新・冪等性を再現して切り分ける

toggle・原子的更新・冪等性の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: single statement
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: atomic update
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: owner condition
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: POSTとredirect

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	single statement	single statement — 入力と前提を確認し、境界を曖昧にしない。
2	atomic update	atomic update — 値がどの順で変換されるかを追跡する。
3	owner condition	owner condition — 失敗を再現し、型またはログから原因を特定する。
4	POSTとredirect	POSTとredirect — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 競合を避けるSQL一文と、再試行時に望む意味を別々に設計します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

done=trueを設定する冪等UPDATEと、二回実行してもtrueのtestを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
UPDATE notes SET done=?1 WHERE id=?2 AND user_id=?3 AND done<>?1 RETURNING done;  
// 0 rowsでも既に期待状態ならsuccessとして扱う設計
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 競合を避けるSQL一文と、再試行時に望む意味を別々に設計します。
- ✓ toggle・原子的更新・冪等性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

削除・確認・hard/soft delete

UNIT 06 · CONCEPT

削除・確認・hard/soft delete

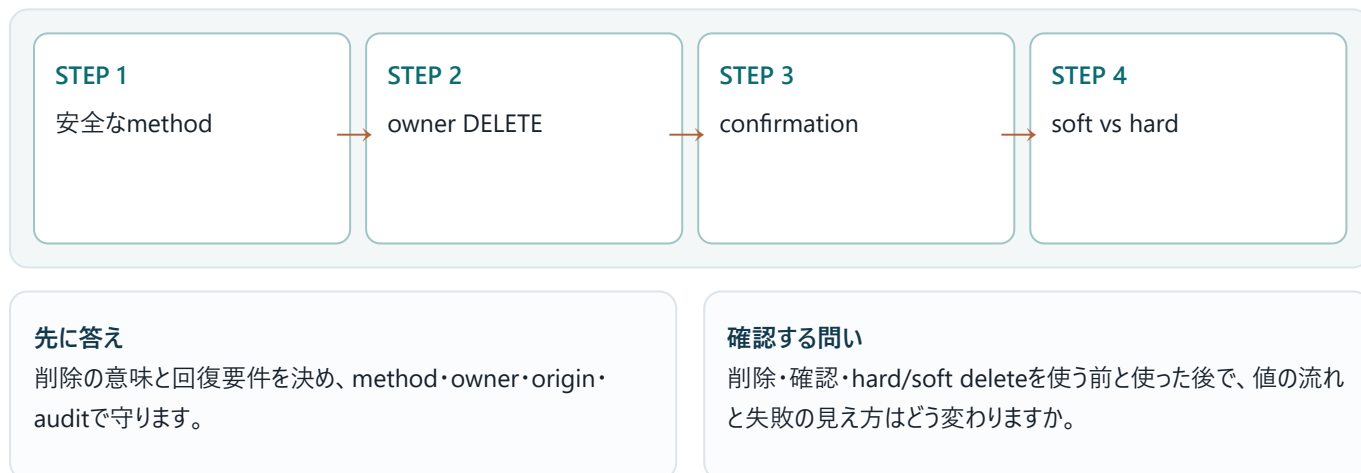
削除はGETではなくPOST/DELETEで行い、owner条件を含めます。誤操作回復や監査が必要ならdeleted_atのsoft delete、法的消去なら関連データを含むhard deleteを選びます。

この単元の到達点

- 安全なmethod
- owner DELETE
- confirmation
- soft vs hard

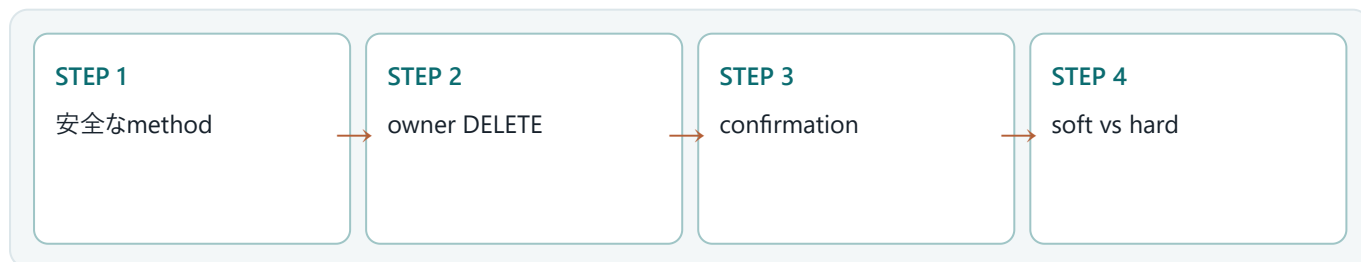
なぜバックエンドで必要か

確認dialogだけを安全策にせず、server側の認証・認可・origin検証・監査を実施します。



UNIT 06 · MENTAL MODEL

削除・確認・hard/soft deleteを図でつかむ



削除はGETではなくPOST/DELETEで行い、owner条件を含めます。誤操作回復や監査が必要ならdeleted_atのsoft delete、法的消去なら関連データを含むhard deleteを選びます。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	安全なmethod	安全なmethod — 入力と前提を確認し、境界を曖昧にしない。
2	owner DELETE	owner DELETE — 値がどの順で変換されるかを追跡する。
3	confirmation	confirmation — 失敗を再現し、型またはログから原因を特定する。
4	soft vs hard	soft vs hard — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

削除・確認・hard/soft deleteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE
まず動く最小コード

削除・確認・hard/soft deleteだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[route(POST "/notes/{note_id}/delete")]async fn delete(cx:&Cx)->Result<SeeOther>{let user=require_user(cx).await?;let id=note_id(cx)?;if repo(cx).delete_owned(user.id,id).await?==0{return Err(not_found().into())}Ok(see_other("/notes"))}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[route(POST "/notes/{note_id}/delete")]async fn delete(cx:&Cx)->Result<Se</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
削除・確認・hard/soft delete	入力、処理、出力を分離して読みます。
削除・確認・hard/soft delete	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力が安全なmethodの条件を満たすか確認する。
- ✓ owner DELETEを実行し、途中の型と状態を追う。
- ✓ confirmationで失敗を成功経路から分離する。
- ✓ soft vs hardにより、出力と副作用を検証する。

型を声に出す 削除・確認・hard/soft deleteに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

削除・確認・hard/soft delete

削除ボタンは対象titleを含む確認を出し、完了後に一覧から消えます。keyboardでも操作できます。

観察ポイント

- 安全なmethod
- owner DELETE
- confirmation

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
UPDATE notes SET deleted_at=CURRENT_TIMESTAMP WHERE id=?1 AND user_id=?2 AND deleted_at IS NULL;
SELECT ... FROM notes WHERE user_id=?1 AND deleted_at IS NULL;
DELETE FROM notes WHERE deleted_at < datetime('now','-30 days');
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06・BACKEND APPLICATION

バックエンドのどこで使うか

確認dialogだけを安全策にせず、server側の認証・認可・origin検証・監査を実施します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	安全なmethodに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	owner DELETEの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	confirmationまたはsoft vs hardの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

削除・確認・hard/soft deleteを再現して切り分ける

削除・確認・hard/soft deleteの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 安全なmethod
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: owner DELETE
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: confirmation
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: soft vs hard

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	安全なmethod	安全なmethod — 入力と前提を確認し、境界を曖昧にしない。
2	owner DELETE	owner DELETE — 値がどの順で変換されるかを追跡する。
3	confirmation	confirmation — 失敗を再現し、型またはログから原因を特定する。
4	soft vs hard	soft vs hard — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 削除の意味と回復要件を決め、method・owner・origin・auditで守ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

soft deleteした行を通常一覧から除外し、30日後purgeするqueryを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
UPDATE notes SET deleted_at=CURRENT_TIMESTAMP WHERE id=?1 AND user_id=?2 AND deleted_at IS NULL;  
SELECT ... FROM notes WHERE user_id=?1 AND deleted_at IS NULL;  
DELETE FROM notes WHERE deleted_at < datetime('now', '-30 days');
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 削除の意味と回復要件を決め、method・owner・origin・auditで守ります。
- ✓ 削除・確認・hard/soft deleteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

検索・filter・並び替え

UNIT 07 · CONCEPT

検索・filter・並び替え

query parameterをtyped structへparseし、許可したsortとstatusだけをenumへ変換します。検索語はbindし、LIKE wildcardを利用者が意図しない場合はescapeします。

この単元の到達点

- typed query
- allowlist sort
- LIKE escape
- URLに状態

なぜバックエンドで必要か

ORDER BY列名をbindできないため、入力をallowlist enumへ変換して固定SQLを選びます。文字列連結しません。



UNIT 07 · MENTAL MODEL

検索・filter・並び替えを図でつかむ



query parameterをtyped structへparseし、許可したsortとstatusだけをenumへ変換します。検索語はbindし、LIKE wildcardを利用者が意図しない場合はescapeします。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	typed query	typed query — 入力と前提を確認し、境界を曖昧にしない。
2	allowlist sort	allowlist sort — 値がどの順で変換されるかを追跡する。
3	LIKE escape	LIKE escape — 失敗を再現し、型またはログから原因を特定する。
4	URLに状態	URLに状態 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

検索・filter・並び替えとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE まず動く最小コード

検索・filter・並び替えだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[query_params(error=bad_request)]struct
NotesQuery{q:Option<String>,status:Option<String>,sort:Option<String>}
let sort=match query.sort.as_deref(){Some("oldest")=>Sort::Oldest,_=>Sort::Newest};
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[query_params(error=bad_request)]struct NotesQuery{q:Option<String>,statu</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>let sort=match query.sort.as_deref() {Some("oldest")=>Sort::Oldest,_=>Sort:</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
検索・filter・並び替え	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がtyped queryの条件を満たすか確認する。
- ✓ allowlist sortを実行し、途中の型と状態を追う。
- ✓ LIKE escapeで失敗を成功経路から分離する。
- ✓ URLに状態により、出力と副作用を検証する。

型を声に出す 検索・filter・並び替えに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

検索・filter・並び替え

検索語、状態filter、並び順がformとURLに残り、0件時は条件を外すlinkが表示されます。

観察ポイント

- typed query
- allowlist sort
- LIKE escape

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
enum Sort{Newest,Oldest,Title}fn parse_sort(v:Option<&str>)->Result<Sort,&'static str>{match v.unwrap_or("newest"){ "newest"=>Ok(Sort::Newest), "oldest"=>Ok(Sort::Oldest), "title"=>Ok(Sort::Title), _=>Err("invalid sort")}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

ORDER BY列名をbindできないため、入力をallowlist enumへ変換して固定SQLを選びます。文字列連結しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	typed queryに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	allowlist sortの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	LIKE escapeまたはURLに状態の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

検索・filter・並び替えを再現して切り分ける

検索・filter・並び替えの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: typed query
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: allowlist sort
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: LIKE escape
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: URLに状態

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	typed query	typed query — 入力と前提を確認し、境界を曖昧にしない。
2	allowlist sort	allowlist sort — 値がどの順で変換されるかを追跡する。
3	LIKE escape	LIKE escape — 失敗を再現し、型またはログから原因を特定する。
4	URLに状態	URLに状態 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 検索状態をURLへ置き、sortはallowlist、値はbindしてSQL injectionを防ぎます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

newest/oldest/title以外のsortを400にするparse関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
enum Sort{Newest,Oldest,Title}fn parse_sort(v:Option<&str>)->Result<Sort,&'static str>{match
v.unwrap_or("newest")
{"newest"=>Ok(Sort::Newest),"oldest"=>Ok(Sort::Oldest),"title"=>Ok(Sort::Title),_=>Err("invalid sort")}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 検索状態をURLへ置き、sortはallowlist、値はbindしてSQL injectionを防ぎます。
- ✓ 検索・filter・並び替えとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

pagination・cursor・件数

UNIT 08・CONCEPT

pagination・cursor・件数

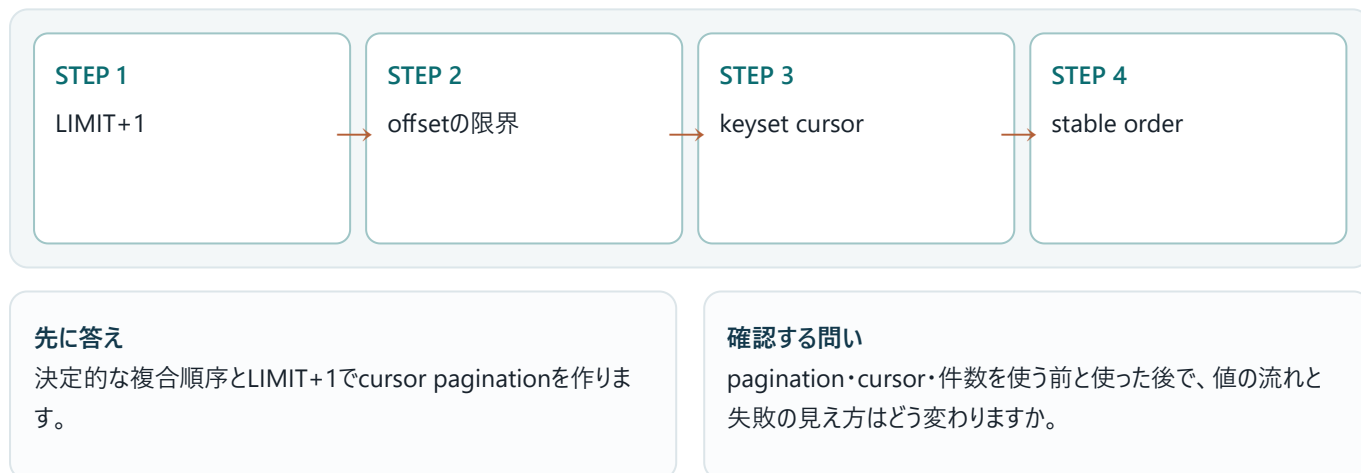
OFFSETは簡単ですが深いpageで遅く更新中に重複・欠落しやすいです。created_atとidのcursor paginationは決定的順序を使い、次cursorをopaqueにします。

この単元の到達点

- LIMIT+1
- offsetの限界
- keyset cursor
- stable order

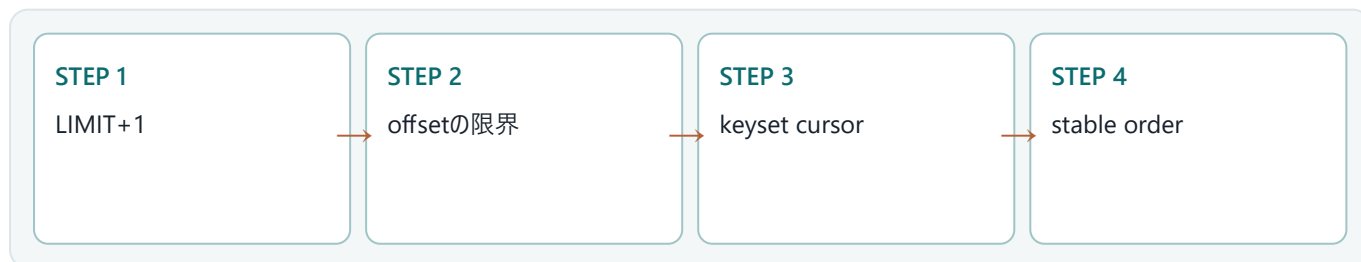
なぜバックエンドで必要か

cursorを利用者が改変してもowner条件は必須です。cursor parse失敗は400、過去cursorは空listとして安全に返します。



UNIT 08・MENTAL MODEL

pagination・cursor・件数を図でつかむ



OFFSETは簡単ですが深いpageで遅く更新中に重複・欠落しやすいです。created_atとidのcursor paginationは決定的順序を使い、次cursorをopaqueにします。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	LIMIT+1	LIMIT+1 — 入力と前提を確認し、境界を曖昧にしない。
2	offsetの限界	offsetの限界 — 値がどの順で変換されるかを追跡する。
3	keyset cursor	keyset cursor — 失敗を再現し、型またはログから原因を特定する。
4	stable order	stable order — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

pagination・cursor・件数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE まず動く最小コード

pagination・cursor・件数だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
SELECT id,title,created_at FROM notes WHERE user_id=?1 AND (created_at<?2 OR(created_at=?2 AND id<?3)) ORDER BY created_at DESC,id DESC LIMIT 21;
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
SELECT id,title,created_at FROM notes WHERE user_id=?1 AND (created_at<?2	失敗時は早期returnし、呼び出し元へエラーを伝播します。

コード／場所	役割と理由
pagination・cursor・件数	入力、処理、出力を分離して読みます。
pagination・cursor・件数	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がLIMIT+1の条件を満たすか確認する。
- ✓ offsetの限界を実行し、途中の型と状態を追う。
- ✓ keyset cursorで失敗を成功経路から分離する。
- ✓ stable orderにより、出力と副作用を検証する。

型を声に出す pagination・cursor・件数に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

pagination・cursor・件数

20件のcardの下に次へlinkがあり、戻る操作でもURL cursorから同じ位置を復元できます。

観察ポイント

- LIMIT+1
- offsetの限界
- keyset cursor

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn page<T>(mut rows:Vec<T>,limit:usize)->(Vec<T>,bool){let has_next=rows.len()>limit;if
has_next{rows.truncate(limit)}(rows,has_next)}fn main(){let(items,next)=page((0..21).collect::<Vec<_>>
(),20);assert_eq!(items.len(),20);assert!(next);}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

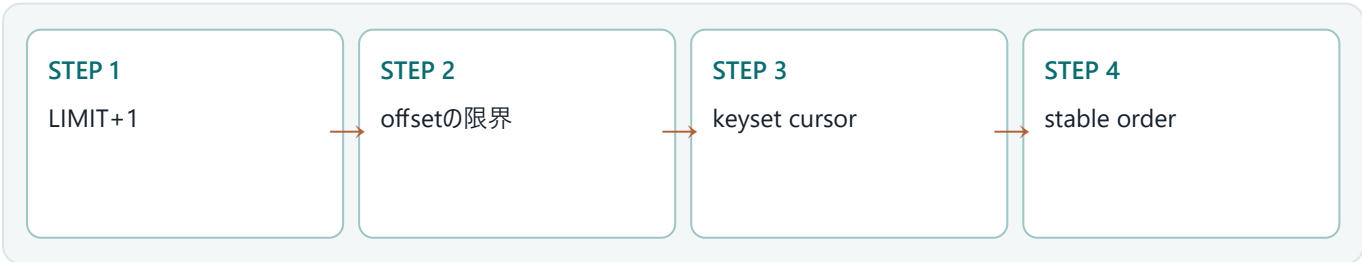
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

cursorを利用者が改変してもowner条件は必須です。cursor parse失敗は400、過去cursorは空listとして安全に返します。



リクエスト側

- LIMIT+1を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- offsetの限界を小さな責務へ分割
- keyset cursorをResultとログで表現
- stable orderをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	LIMIT+1に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	offsetの限界の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	keyset cursorまたはstable orderの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

pagination・cursor・件数を再現して切り分ける

pagination・cursor・件数の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: LIMIT+1
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: offsetの限界
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: keyset cursor
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: stable order

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	LIMIT+1	LIMIT+1 — 入力と前提を確認し、境界を曖昧にしない。
2	offsetの限界	offsetの限界 — 値がどの順で変換されるかを追跡する。
3	keyset cursor	keyset cursor — 失敗を再現し、型またはログから原因を特定する。
4	stable order	stable order — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 決定的な複合順序とLIMIT+1でcursor paginationを作ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

21件取得からhas_nextと20件itemsを作るPage関数を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn page<T>(mut rows:Vec<T>,limit:usize)->(Vec<T>,bool){let has_next=rows.len()>limit;if
has_next{rows.truncate(limit)}(rows,has_next)}fn main(){let(items,next)=page((0..21).collect::<Vec<_>>
(),20);assert_eq!(items.len(),20);assert!(next);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 決定的な複合順序とLIMIT+1でcursor paginationを作ります。
- ✓ pagination・cursor・件数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

flash・error summary・PRG後の通知

UNIT 09・CONCEPT

flash・error summary・PRG後の通知

redirect後の成功通知は一回限りのflashとしてsessionまたは短命cookieへ保存し、次GETで取り出して削除します。errorはredirectせず400で同じformへ表示します。

この単元の到達点

- one-time flash
- success通知
- 400 inline error
- focus management

なぜバックエンドで必要か

flashへ利用者入力や秘密をそのまま入れず、server側の定型messageとevent codeを使います。



UNIT 09・MENTAL MODEL

flash・error summary・PRG後の通知を図でつかむ



redirect後の成功通知は一回限りのflashとしてsessionまたは短命cookieへ保存し、次GETで取り出して削除します。errorはredirectせず400で同じformへ表示します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	one-time flash	one-time flash — 入力と前提を確認し、境界を曖昧にしない。
2	success通知	success通知 — 値がどの順で変換されるかを追跡する。
3	400 inline error	400 inline error — 失敗を再現し、型またはログから原因を特定する。
4	focus management	focus management — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

flash・error summary・PRG後の通知とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE まず動く最小コード

flash・error summary・PRG後の通知だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
POST create: flash(cx, "ノートを作成しました"); Ok(see_other("/notes"))
GET notes: let message=take_flash(cx); view!{if let Some(m)=message{<div role="status">(m)</div>}}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
POST create: flash(cx, "ノートを作成しました"); Ok(see_other("/notes"))	この行が受け取る値、作る値、副作用の有無を確認します。

コード／場所	役割と理由
GET notes: let message=take_flash(cx); view!{if let Some(m)=message{<div r	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
flash・error summary・PRG後の通知	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がone-time flashの条件を満たすか確認する。
- ✓ success通知を実行し、途中の型と状態を追う。
- ✓ 400 inline errorで失敗を成功経路から分離する。
- ✓ focus managementにより、出力と副作用を検証する。

型を声に出す flash・error summary・PRG後の通知に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

flash・error summary・PRG後の通知

作成・更新・削除後はページ上部に一度だけ成功通知、validation失敗は各入力の近くに表示されます。

観察ポイント

- one-time flash
- success通知
- 400 inline error

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
delete成功: 303後の別GETなのでflash role=status
login失敗: 入力formと同じrequestで400 inline role=alert
password値は保持しない
次focusはerror summaryまたはmain heading
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

flashへ利用者入力や秘密をそのまま入れず、server側の定型messageとevent codeを使います。



リクエスト側

- one-time flashを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- success通知を小さな責務へ分割
- 400 inline errorをResultとログで表現
- focus managementをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	one-time flashに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	success通知の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	400 inline errorまたはfocus managementの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

flash・error summary・PRG後の通知を再現して切り分ける

flash・error summary・PRG後の通知の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: one-time flash
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: success通知
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 400 inline error
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: focus management

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	one-time flash	one-time flash — 入力と前提を確認し、境界を曖昧にしない。
2	success通知	success通知 — 値がどの順で変換されるかを追跡する。
3	400 inline error	400 inline error — 失敗を再現し、型またはログから原因を特定する。
4	focus management	focus management — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 成功はPRG後の一回flash、入力errorは400の同画面再表示に分けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

delete成功とlogin失敗で、どちらをflash/inlineにするか理由と実装を示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
delete成功: 303後の別GETなのでflash role=status  
login失敗: 入力formと同じrequestで400 inline role=alert  
password値は保持しない  
次focusはerror summaryまたはmain heading
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 成功はPRG後の一回flash、入力errorは400の同画面再表示に分けます。
- ✓ flash・error summary・PRG後の通知とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

responsive UI・a11y・総合CRUD試験

UNIT 10・CONCEPT

responsive UI・a11y・総合CRUD試験

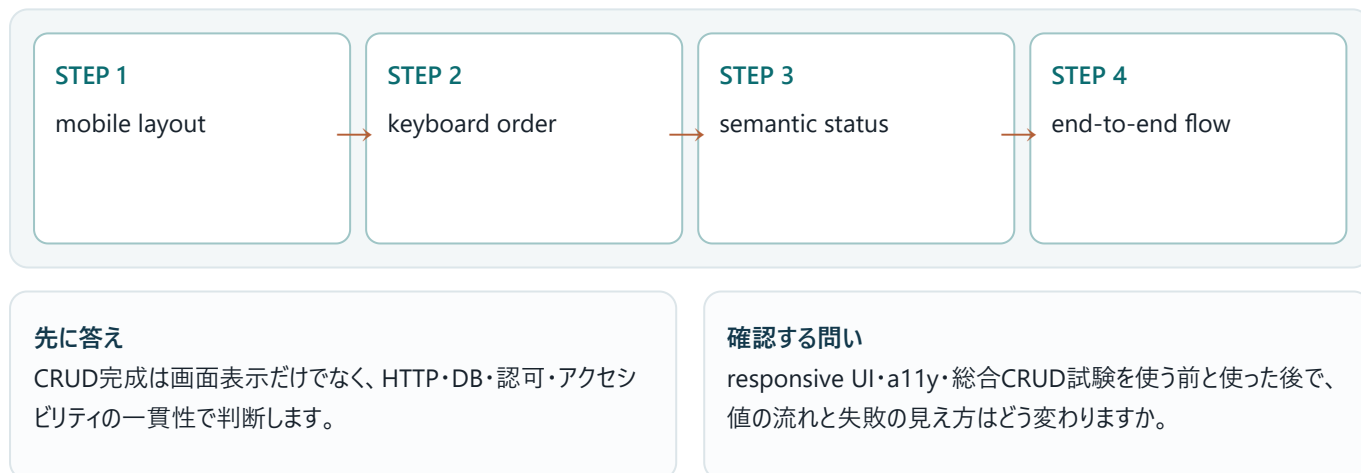
CRUD画面をmobile、keyboard、screen reader、高contrastで確認します。見た目のscreenshotだけでなくHTTP、DB、認可、focus、accessible nameまでe2eで検証します。

この単元の到達点

- mobile layout
- keyboard order
- semantic status
- end-to-end flow

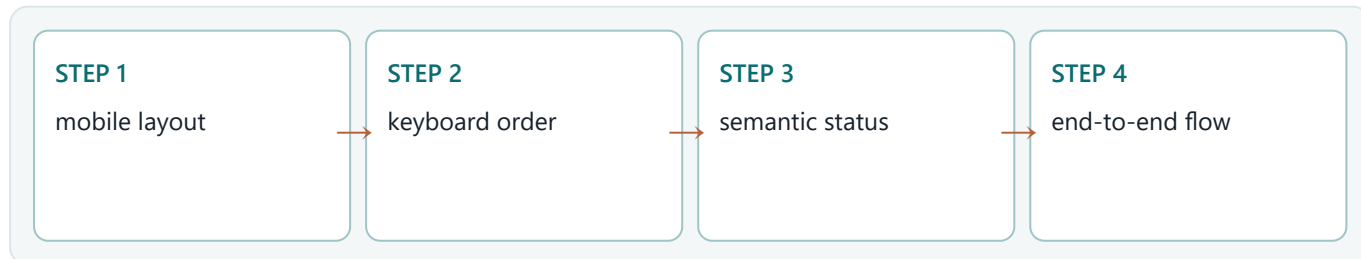
なぜバックエンドで必要か

e2eはregister→login→create→edit→toggle→search→delete→logoutを一通し、別userのID改変も拒否します。



UNIT 10・MENTAL MODEL

responsive UI・a11y・総合CRUD試験を図でつかむ



CRUD画面をmobile、keyboard、screen reader、高contrastで確認します。見た目のscreenshotだけでなくHTTP、DB、認可、focus、accessible nameまでe2eで検証します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	mobile layout	mobile layout — 入力と前提を確認し、境界を曖昧にしない。
2	keyboard order	keyboard order — 値がどの順で変換されるかを追跡する。
3	semantic status	semantic status — 失敗を再現し、型またはログから原因を特定する。
4	end-to-end flow	end-to-end flow — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

responsive UI・a11y・総合CRUD試験とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

responsive UI・a11y・総合CRUD試験だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
@media(max-width:42rem){.note-card{grid-template-columns:1fr}.actions{display:grid;grid-template-columns:1fr 1fr}}.visually-hidden{position:absolute;width:1px;height:1px;overflow:hidden;clip:rect(0,0,0,0)}
```

●●● 実行コマンド

```
cargo test
# browserでE2E smoke
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
@media(max-width:42rem){.note-card{grid-template-columns:1fr}.actions{disp	この行が受け取る値、作る値、副作用の有無を確認します。
.visually-hidden{position:absolute;width:1px;height:1px;overflow:hidden;cl	この行が受け取る値、作る値、副作用の有無を確認します。
responsive UI・a11y・総合CRUD試験	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がmobile layoutの条件を満たすか確認する。
- ✓ keyboard orderを実行し、途中の型と状態を追う。
- ✓ semantic statusで失敗を成功経路から分離する。
- ✓ end-to-end flowにより、出力と副作用を検証する。

型を声に出す responsive UI・a11y・総合CRUD試験に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

responsive UI・a11y・総合CRUD試験

desktopとmobileの両方で全CRUDが操作でき、focus ringと状態messageが明確に見えます。

観察ポイント

- mobile layout
- keyboard order
- semantic status

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
Browser: value/focus/status/card
HTTP: 400/303/404/403 + Location
DB: owner/title/version/done/row deletion
Log: request_id/operation/result/no secret
A11y: label/name/order/contrast
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

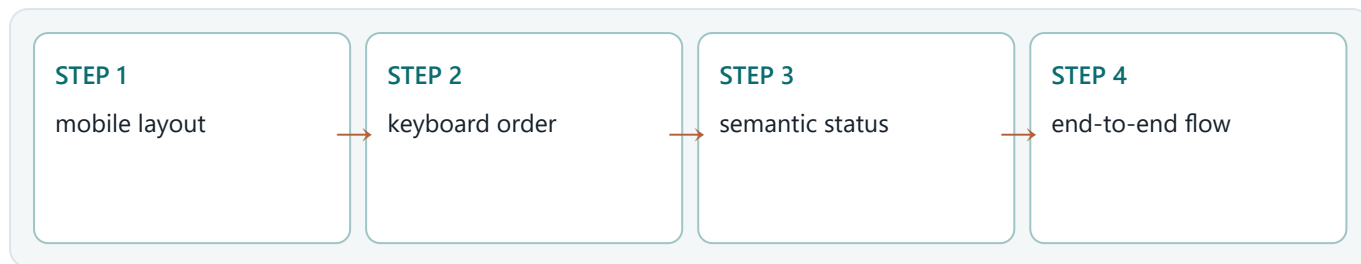
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

e2eはregister→login→create→edit→toggle→search→delete→logoutを一通し、別userのID改変も拒否します。



リクエスト側

- mobile layoutを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- keyboard orderを小さな責務へ分割
- semantic statusをResultとログで表現
- end-to-end flowをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	mobile layoutに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	keyboard orderの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	semantic statusまたはend-to-end flowの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

responsive UI・a11y・総合CRUD試験を再現して切り分ける

responsive UI・a11y・総合CRUD試験の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: mobile layout
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: keyboard order
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: semantic status
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: end-to-end flow

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	mobile layout	mobile layout — 入力と前提を確認し、境界を曖昧にしない。
2	keyboard order	keyboard order — 値がどの順で変換されるかを追跡する。
3	semantic status	semantic status — 失敗を再現し、型またはログから原因を特定する。
4	end-to-end flow	end-to-end flow — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 CRUD完成は画面表示だけでなく、HTTP・DB・認可・アクセシビリティの一貫性で判断します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

CRUD総合試験の観測点をbrowser、HTTP、DB、logに分けて書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
Browser: value/focus/status/card
HTTP: 400/303/404/403 + Location
DB: owner/title/version/done/row deletion
Log: request_id/operation/result/no secret
A11y: label/name/order/contrast
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ CRUD完成は画面表示だけでなく、HTTP・DB・認可・アクセシビリティの一貫性で判断します。
- ✓ responsive UI・a11y・総合CRUD試験とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

RustNoteの認証付きCRUDをbrowserで完走する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	一覧表示・空状態・所有者query	一覧は認証→owner query→空/非空分岐→semantic listの順に実装します。
02	作成form・validation・PRG	副作用前に検証し、失敗は400再表示、成功は303 redirectにします。
03	詳細・path ID・not found	path parse、domain検証、owner queryの三段階でIDを守ります。
04	編集・事前表示・競合	編集はowner条件とversion条件を一つのUPDATEに入れ、競合を409で可視化します。
05	toggle・原子的更新・冪等性	競合を避けるSQL一文と、再試行時に望む意味を別々に設計します。
06	削除・確認・hard/soft delete	削除の意味と回復要件を決め、method・owner・origin・auditで守ります。
07	検索・filter・並び替え	検索状態をURLへ置き、sortはallowlist、値はbindしてSQL injectionを防ぎます。
08	pagination・cursor・件数	決定的な複合順序とLIMIT+1でcursor paginationを作ります。
09	flash・error summary・PRG後の通知	成功はPRG後の一回flash、入力errorは400の同画面再表示に分けます。
10	responsive UI・a11y・総合CRUD試験	CRUD完成は画面表示だけでなく、HTTP・DB・認可・アクセシビリティの一貫性で判断します。

完了条件

- ✓ 他人IDでshow/update/deleteできない
- ✓ 重複POSTと編集競合を扱う
- ✓ mobileとkeyboardで全操作可能
- ✓ 各操作のHTTP/DB/log結果を説明できる

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 一覧表示・空状態・所有者queryを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. 作成form・validation・PRGを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. 詳細・path ID・not foundを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. 編集・事前表示・競合を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. toggle・原子的更新・冪等性を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. 削除・確認・hard/soft deleteを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. 検索・filter・並び替えを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. pagination・cursor・件数を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. flash・error summary・PRG後の通知を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. responsive UI・a11y・総合CRUD試験を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 一覧表示・空状態・所有者query

一覧表示・空状態・所有者queryとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 一覧は認証→owner query→空/非空分岐→semantic listの順に実装します。

02. 作成form・validation・PRG

作成form・validation・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 副作用前に検証し、失敗は400再表示、成功は303 redirectにします。

03. 詳細・path ID・not found

詳細・path ID・not foundとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: path parse、domain検証、owner queryの三段階でIDを守ります。

04. 編集・事前表示・競合

編集・事前表示・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 編集はowner条件とversion条件を一つのUPDATEに入れ、競合を409で可視化します。

05. toggle・原子的更新・冪等性

toggle・原子的更新・冪等性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 競合を避けるSQL一文と、再試行時に望む意味を別々に設計します。

06. 削除・確認・hard/soft delete

削除・確認・hard/soft deleteとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 削除の意味と回復要件を決め、method・owner・origin・auditで守ります。

07. 検索・filter・並び替え

検索・filter・並び替えとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 検索状態をURLへ置き、sortはallowlist、値はbindしてSQL injectionを防ぎます。

08. pagination・cursor・件数

pagination・cursor・件数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 決定的な複合順序とLIMIT+1でcursor paginationを作ります。

09. flash・error summary・PRG後の通知

flash・error summary・PRG後の通知とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 成功はPRG後の一回flash、入力errorは400の同画面再表示に分けます。

10. responsive UI・a11y・総合CRUD試験

responsive UI・a11y・総合CRUD試験とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: CRUD完成は画面表示だけでなく、HTTP・DB・認可・アクセシビリティの一貫性で判断します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
一覧表示・空状態・所有者 query	一覧は認証→owner query→空/非空分岐→semantic listの順に実装します。	SELECTにuser_idとLIMIT、決定的ORDER BYを含めます。viewへpassword/session型を渡しません。
作成form・validation・PRG	副作用前に検証し、失敗は400再表示、成功は303 redirectにします。	validation前にDBへ触れず、DB CHECKとUNIQUE競合もAppErrorへ変換します。二重送信はPRGと一意キーで抑えます。
詳細・path ID・not found	path parse、domain検証、owner queryの三段階でIDを守ります。	ID parse成功は認可成功を意味しません。DB queryまでowner条件を運び、error bodyにDBや他user情報を出しません。
編集・事前表示・競合	編集はowner条件とversion条件を一つのUPDATEに入れ、競合を409で可視化します。	GET時のversionをhidden fieldへ持たせませんが、user_idはhidden値を信用せずsessionから取ります。
toggle・原子的更新・冪等性	競合を避けるSQL一文と、再試行時に望む意味を別々に設計します。	toggleは冪等ではないためretryで二回実行すると元へ戻ります。明示的なset-done true/false routeやidempotency keyも検討します。
削除・確認・hard/soft delete	削除の意味と回復要件を決め、method・owner・origin・auditで守ります。	確認dialogだけを安全策にせず、server側の認証・認可・origin検証・監査を実施します。
検索・filter・並び替え	検索状態をURLへ置き、sortはallowlist、値はbindしてSQL injectionを防ぎます。	ORDER BY列名をbindできないため、入力をallowlist enumへ変換して固定SQLを選びます。文字列連結しません。
pagination・cursor・件数	決定的な複合順序とLIMIT+1でcursor paginationを作ります。	cursorを利用者が改変してもowner条件は必須です。cursor parse失敗は400、過去cursorは空listとして安全に返します。
flash・error summary・PRG後の通知	成功はPRG後の一回flash、入力errorは400の同画面再表示に分けます。	flashへ利用者入力や秘密をそのまま入れず、server側の定型messageとevent codeを使います。
responsive UI・a11y・総合 CRUD試験	CRUD完成は画面表示だけでなく、HTTP・DB・認可・アクセシビリティの一貫性で判断します。	e2eはregister→login→create→edit→toggle→search→delete→logoutを一通し、別userのID改変も拒否します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。